LAPORAN PRAKTIKUM

POSTTEST 8

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Belva Abhinaya (2609106005)

Kelas (A1 '26)

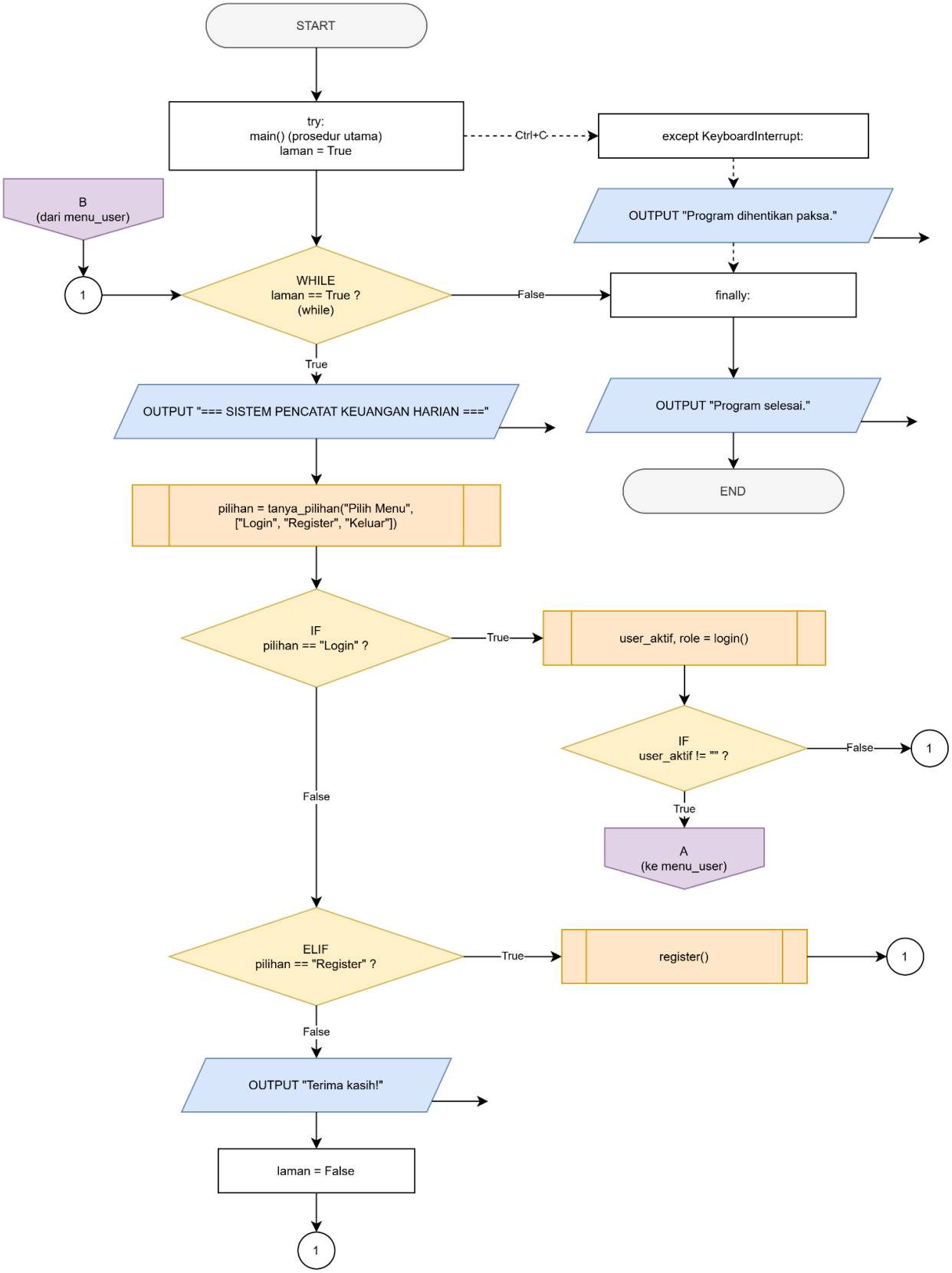
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

1. Flowchart


START
try:
main() (prosedur utama)
laman = True
Ctrl+C
except KeyboardInterrupt:
OUTPUT "Program dihentikan paksa."
B
(dari menu_user)
1
WHILE
laman == True ?
(while)
False
finally:
True
OUTPUT "=== SISTEM PENCATAT KEUANGAN HARIAN ==="
OUTPUT "Program selesai."
END
pilihan = tanya_pilihan("Pilih Menu",
["Login", "Register", "Keluar"])
IF
pilihan == "Login" ?
True
user_aktif, role = login()
IF
user_aktif != "" ?
False
1
True
A
(ke menu_user)
False
ELIF
pilihan == "Register" ?
True
register()
1
False
OUTPUT "Terima kasih!"
laman = False
1

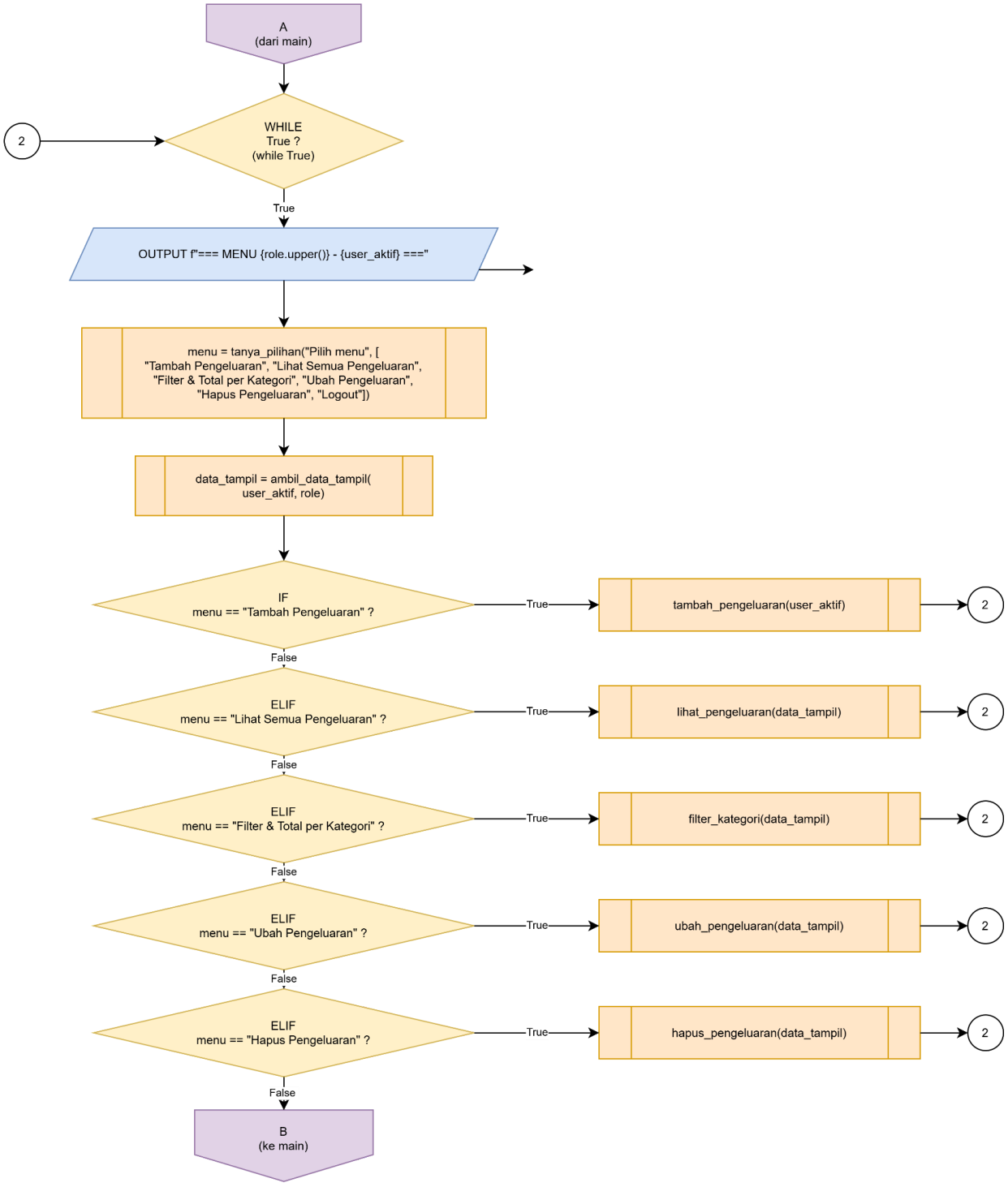
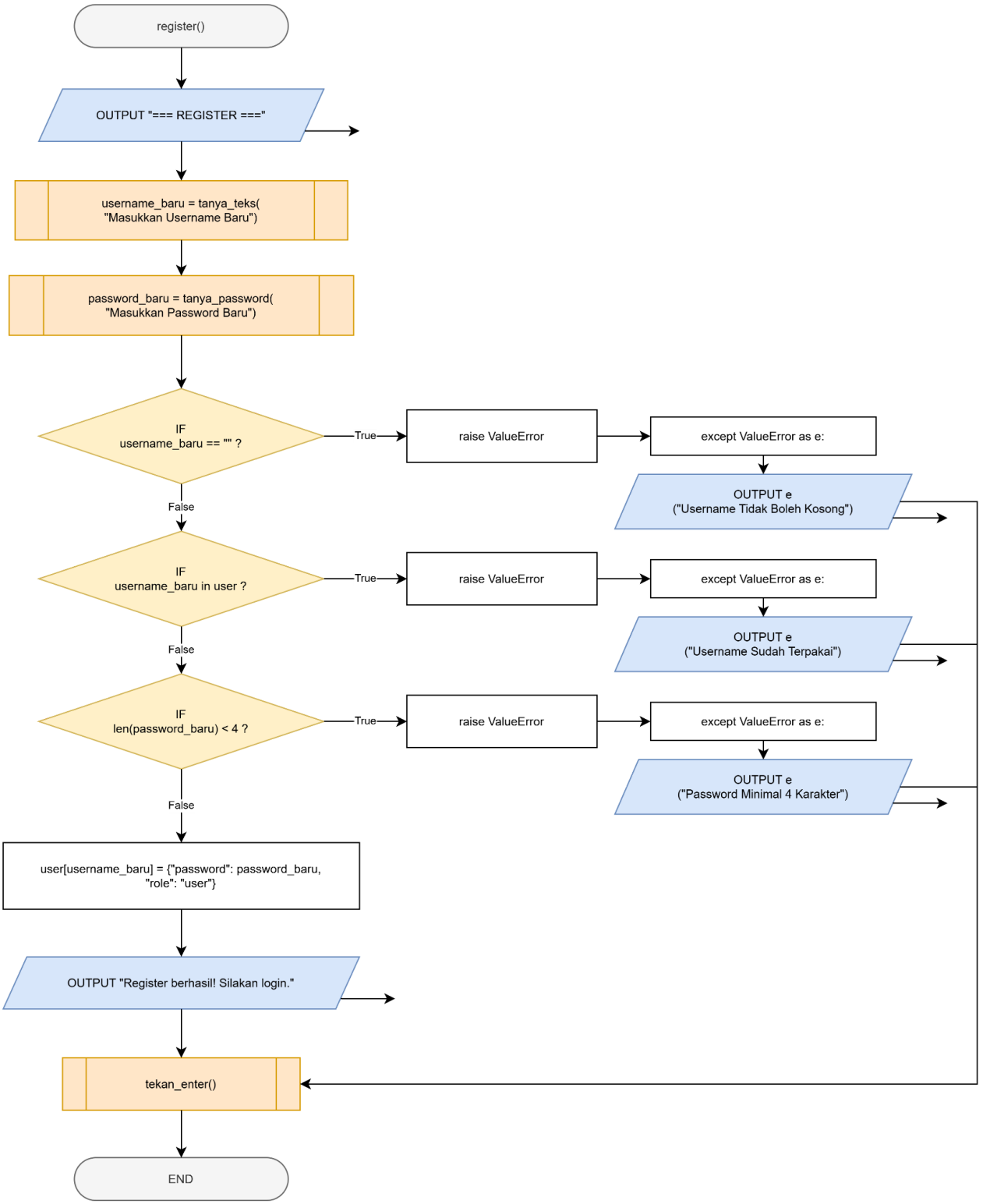


Diagram 1.1. Flowchart main() dan menu_user() (file main.py)


register()
OUTPUT "=== REGISTER ==="
username_baru = tanya_teks(
"Masukkan Username Baru")
password_baru = tanya_password(
"Masukkan Password Baru")
IF
username_baru == "" ?
True
raise ValueError
except ValueError as e:
OUTPUT e
("Username Tidak Boleh Kosong")
False
IF
username_baru in user ?
True
raise ValueError
except ValueError as e:
OUTPUT e
("Username Sudah Terpakai")
False
IF
len(password_baru) < 4 ?
True
raise ValueError
except ValueError as e:
OUTPUT e
("Password Minimal 4 Karakter")
False
user[username_baru] = {"password": password_baru,
"role": "user"}
OUTPUT "Register berhasil! Silakan login."
tekan_enter()
END

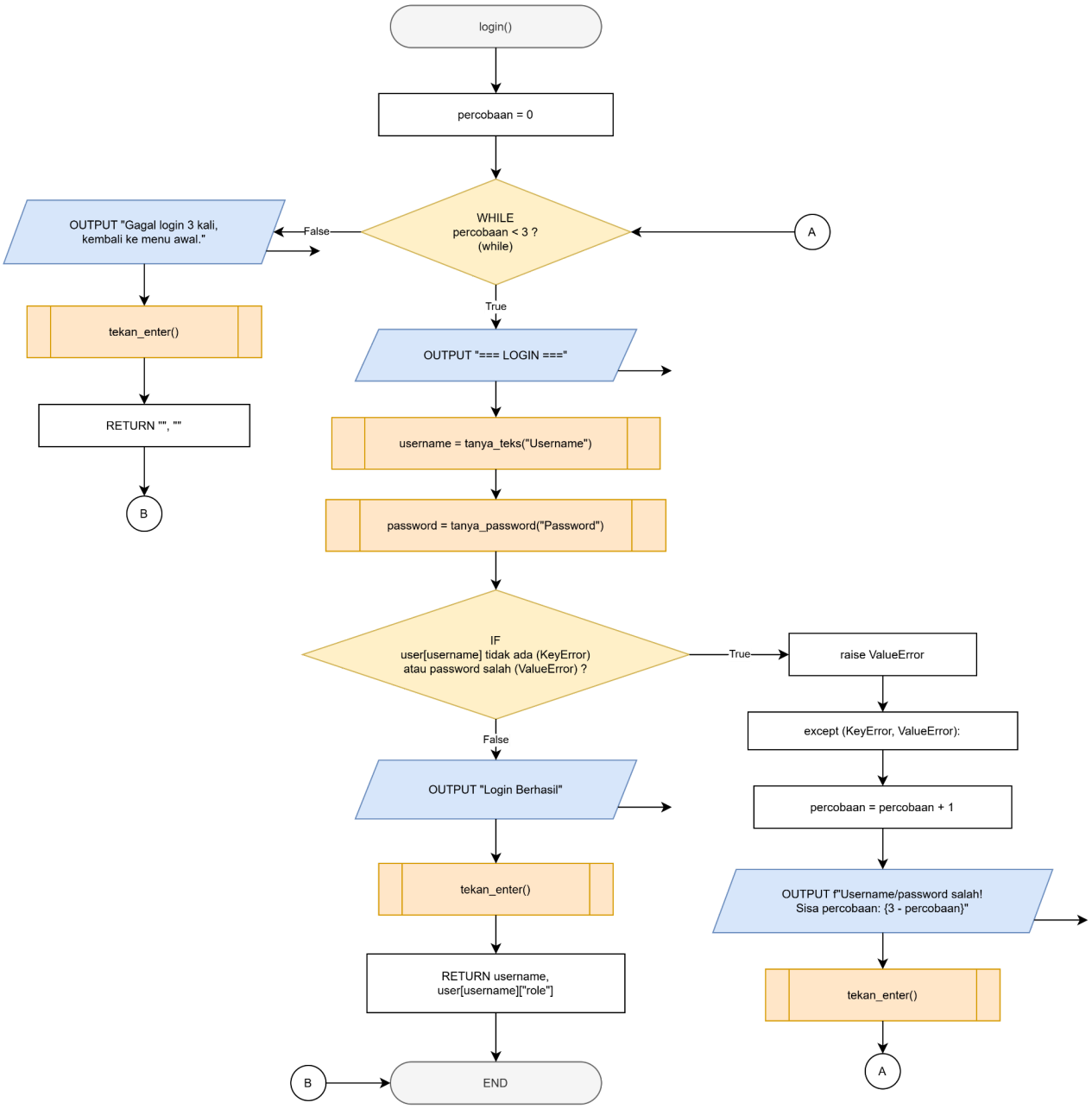


Diagram 1.2. Flowchart register() dan login() (file autentikasi.py)

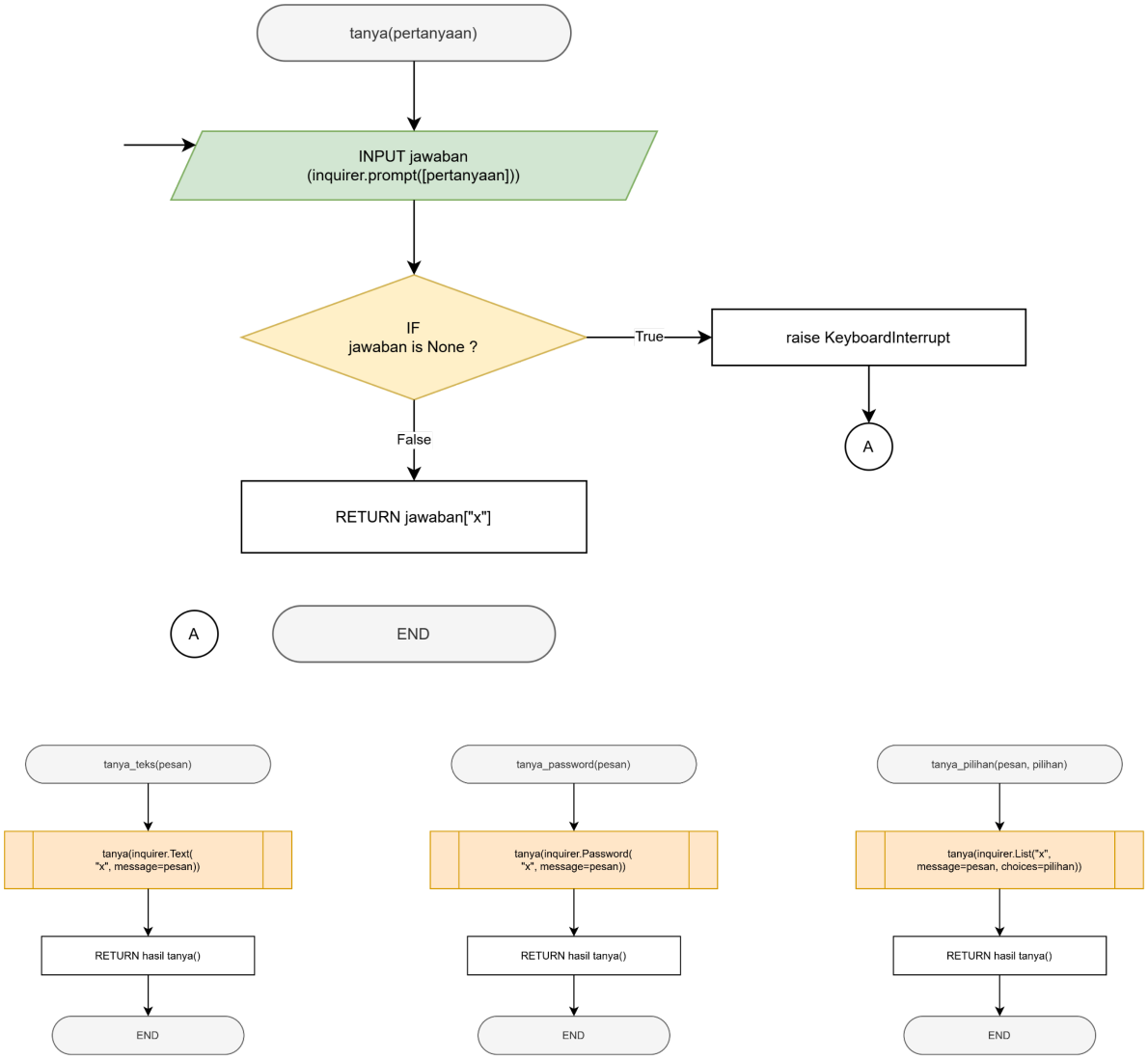
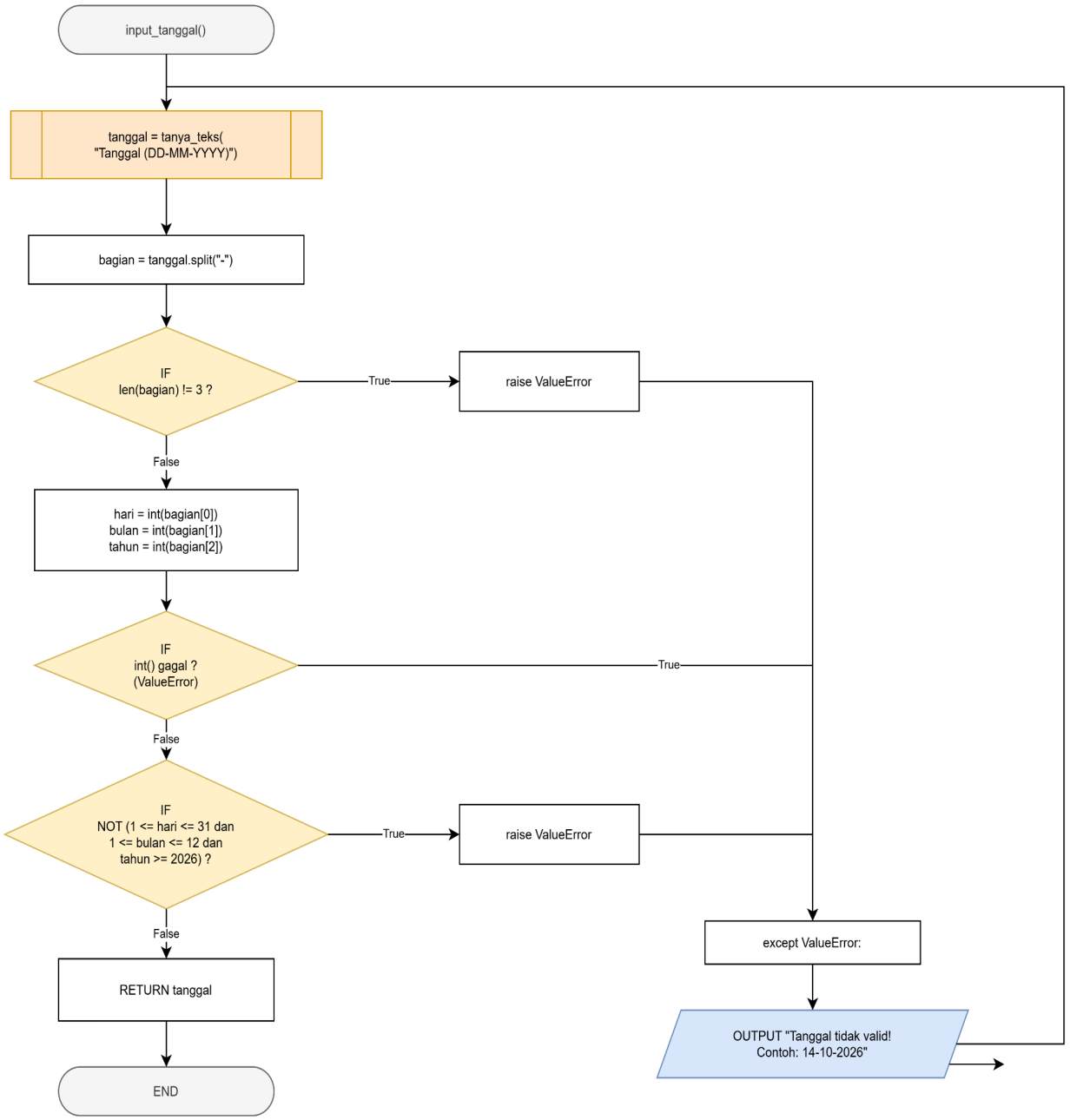


Diagram 1.3. Flowchart tanya(), tanya_teks(), tanya_password(), dan tanya_pilihan() (file fungsi_umum.py)


input_tanggal()
tanggal = tanya_teks(
"Tanggal (DD-MM-YYYY)")
bagian = tanggal.split("-")
IF
len(bagian) != 3 ?
True
raise ValueError
False
hari = int(bagian[0])
bulan = int(bagian[1])
tahun = int(bagian[2])
IF
int() gagal ?
(ValueError)
True
False
IF
NOT (1 <= hari <= 31 dan
1 <= bulan <= 12 dan
tahun >= 2026) ?
True
raise ValueError
False
except ValueError:
RETURN tanggal
OUTPUT "Tanggal tidak valid!
Contoh: 14-10-2026"
END

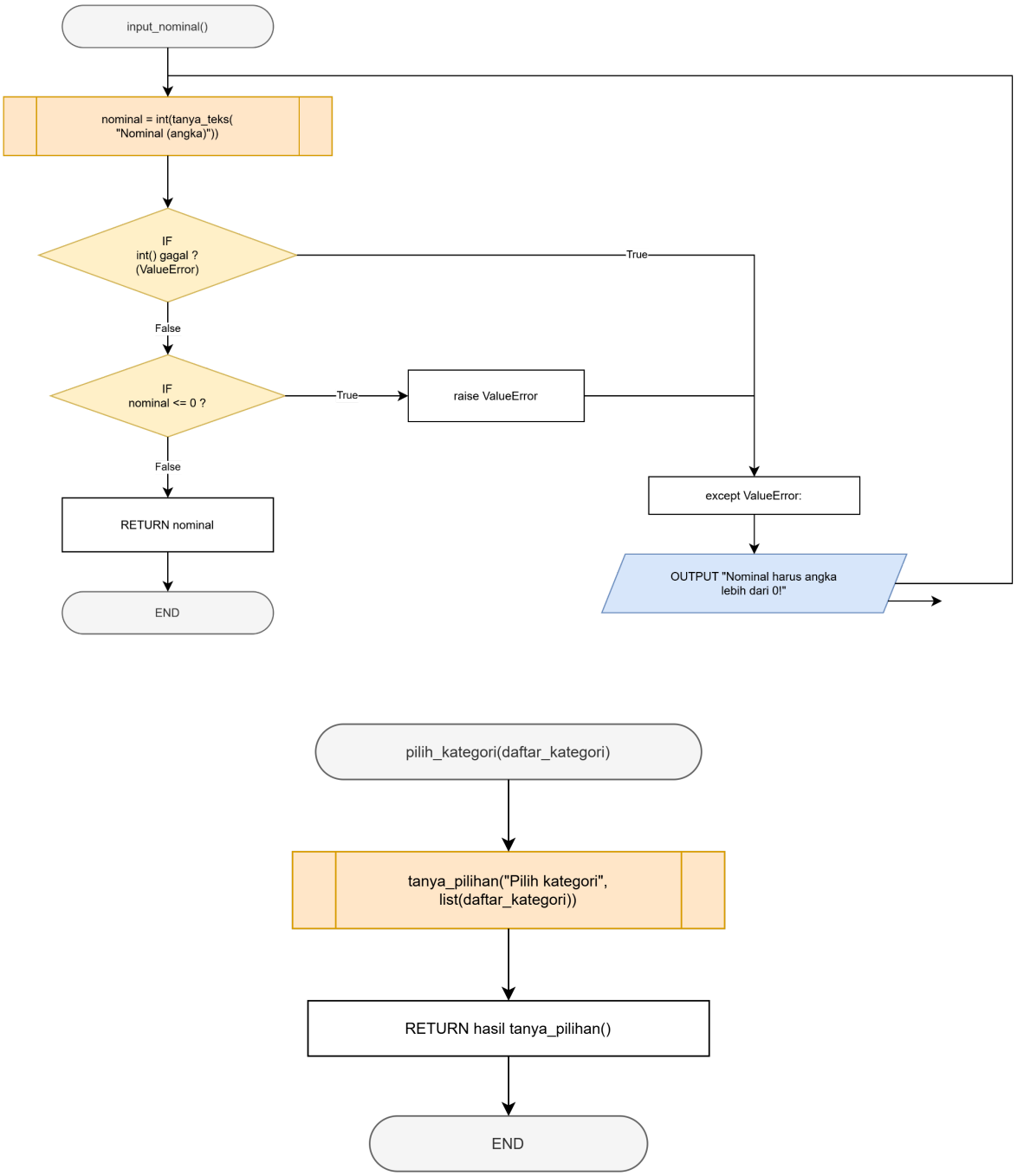
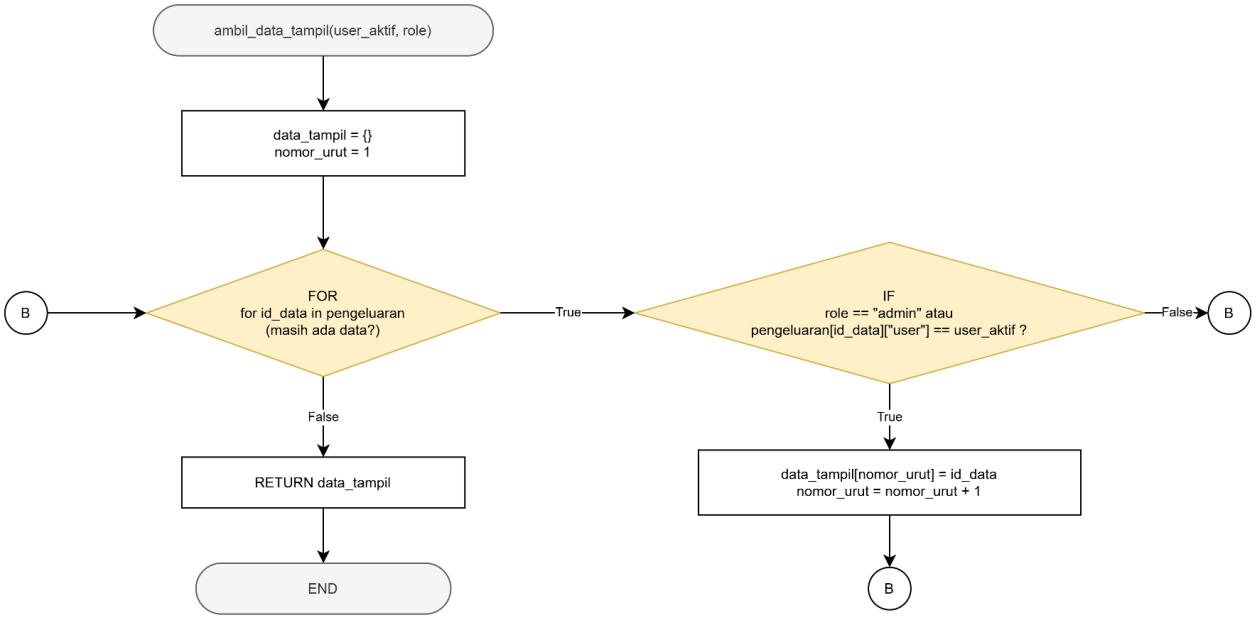
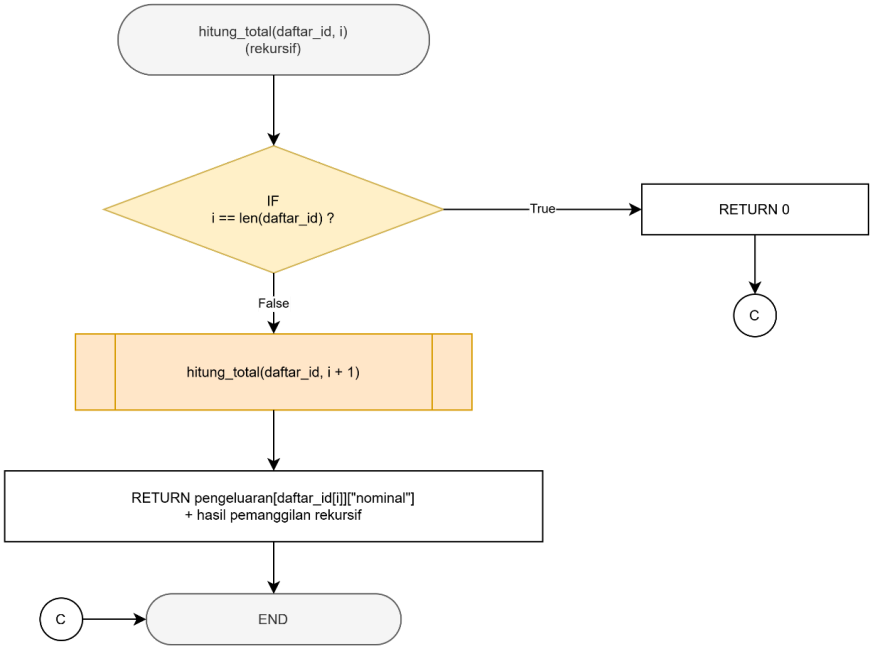


Diagram 1.4. Flowchart input_tanggal(), input_nominal(), dan pilih_kategori() (file fungsi_umum.py)


ambil_data_tampil(user_aktif, role)
data_tampil = {}
nomor_urut = 1
B
FOR
for id_data in pengeluaran
(masih ada data?)
True
IF
role == "admin" atau
pengeluaran[id_data]["user"] == user_aktif ?
False
B
False
True
RETURN data_tampil
data_tampil[nomor_urut] = id_data
nomor_urut = nomor_urut + 1
END
B


hitung_total(daftar_id, i)
(rekursif)
IF
i == len(daftar_id) ?
True
RETURN 0
C
False
hitung_total(daftar_id, i + 1)
RETURN pengeluaran[daftar_id[i]]["nominal"]
+ hasil pemanggilan rekursif
C
END

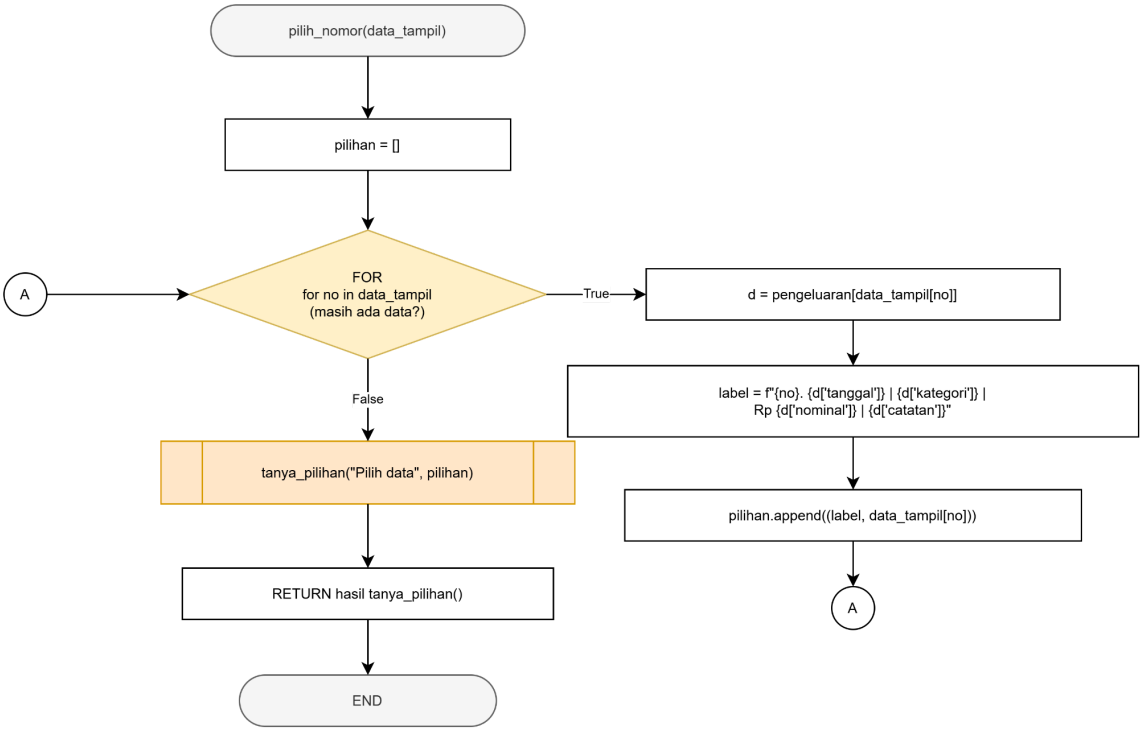


Diagram 1.5. Flowchart ambil_data_tampil(), hitung_total(), dan pilih_nomor() (file fungsi_umum.py)

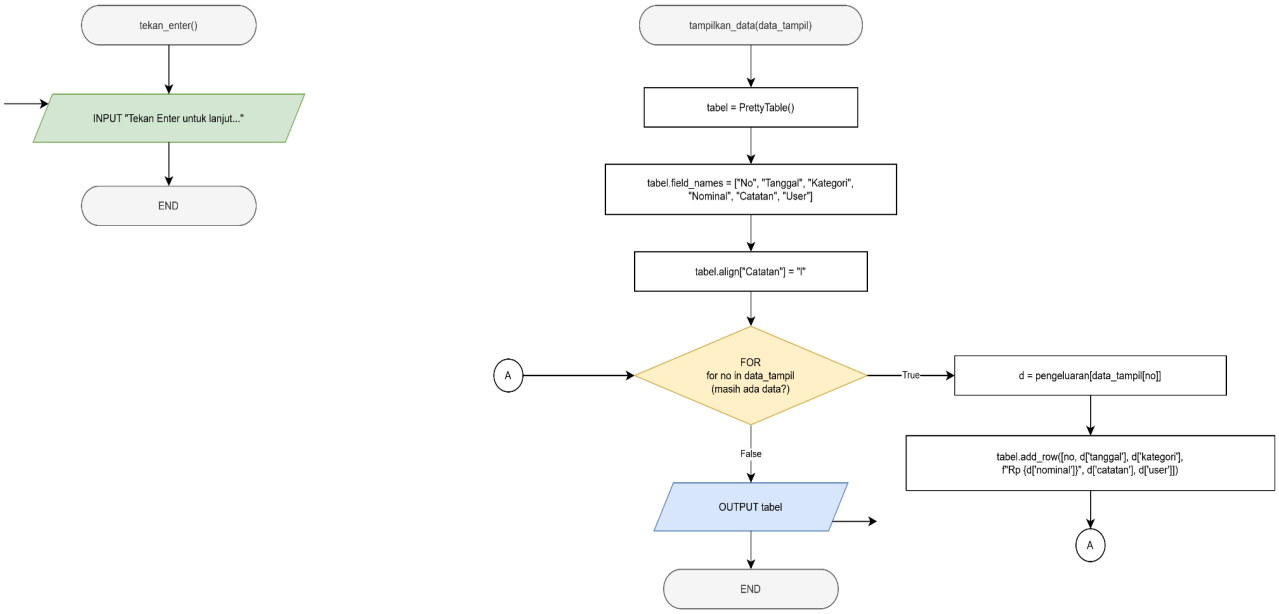
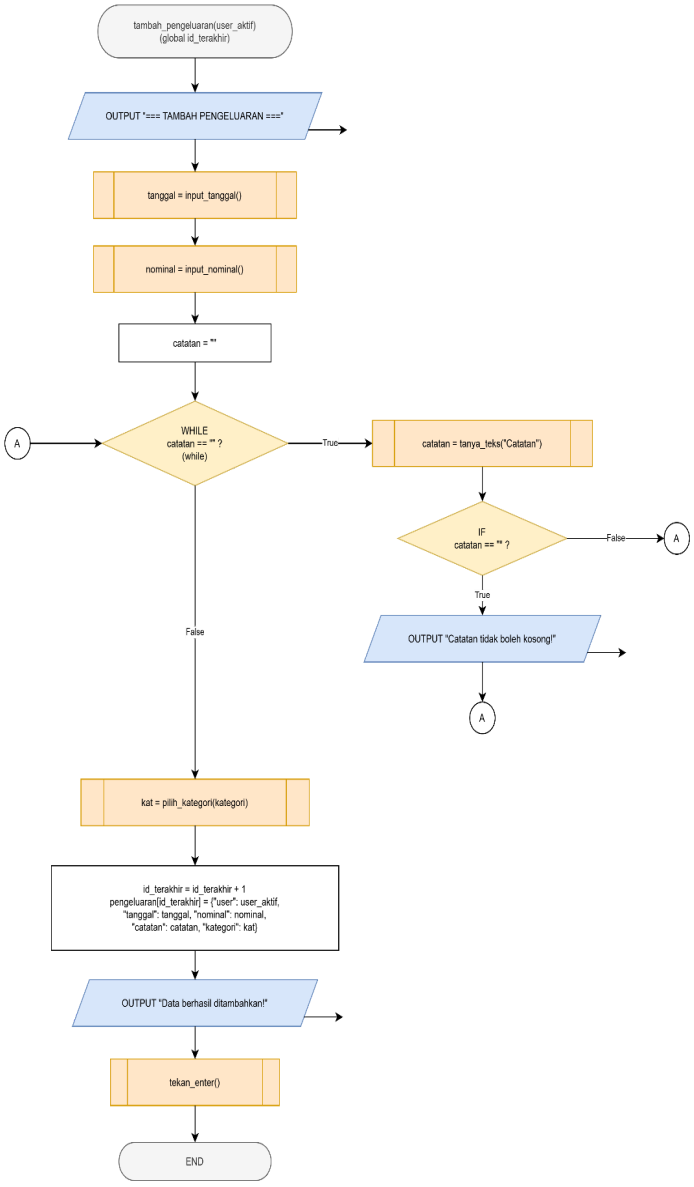
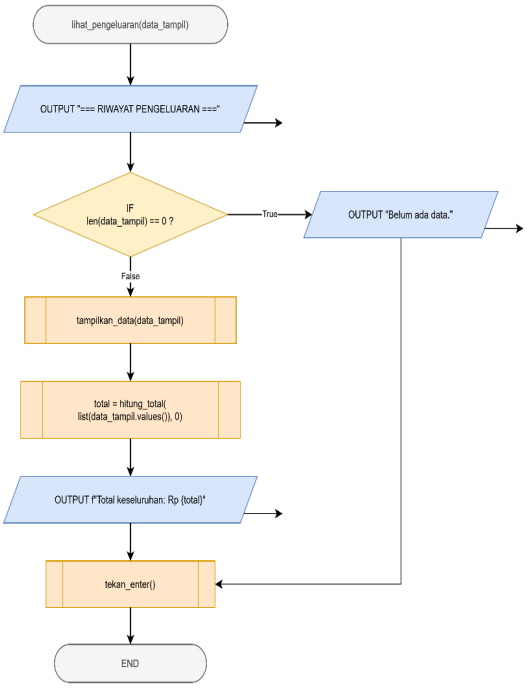


Diagram 1.6. Flowchart tekan_enter() dan tampilkan_data() (file fungsi_umum.py dan tampilan.py)


tambah_pengeluaran(user_aktif)
(global id_terakhir)
OUTPUT "=== TAMBAH PENGELUARAN ==="
tanggal = input_tanggal()
nominal = input_nominal()
catatan = ""
A
WHILE
catatan == "" ?
(while)
True
catatan = tanya_teks("Catatan")
IF
catatan == "" ?
False
A
True
OUTPUT "Catatan tidak boleh kosong!"
A
False
kat = pilih_kategori(kategori)
id_terakhir = id_terakhir + 1
pengeluaran[id_terakhir] = {"user": user_aktif,
"tanggal": tanggal, "nominal": nominal,
"catatan": catatan, "kategori": kat}
OUTPUT "Data berhasil ditambahkan!"
tekan_enter()
END


lihat_pengeluaran(data_tampil)
OUTPUT "=== RIWAYAT PENGELUARAN ==="
IF
len(data_tampil) == 0 ?
True
OUTPUT "Belum ada data."
False
tampilkan_data(data_tampil)
total = hitung_total(
list(data_tampil.values()), 0)
OUTPUT f"Total keseluruhan: Rp {total}"
tekan_enter()
END

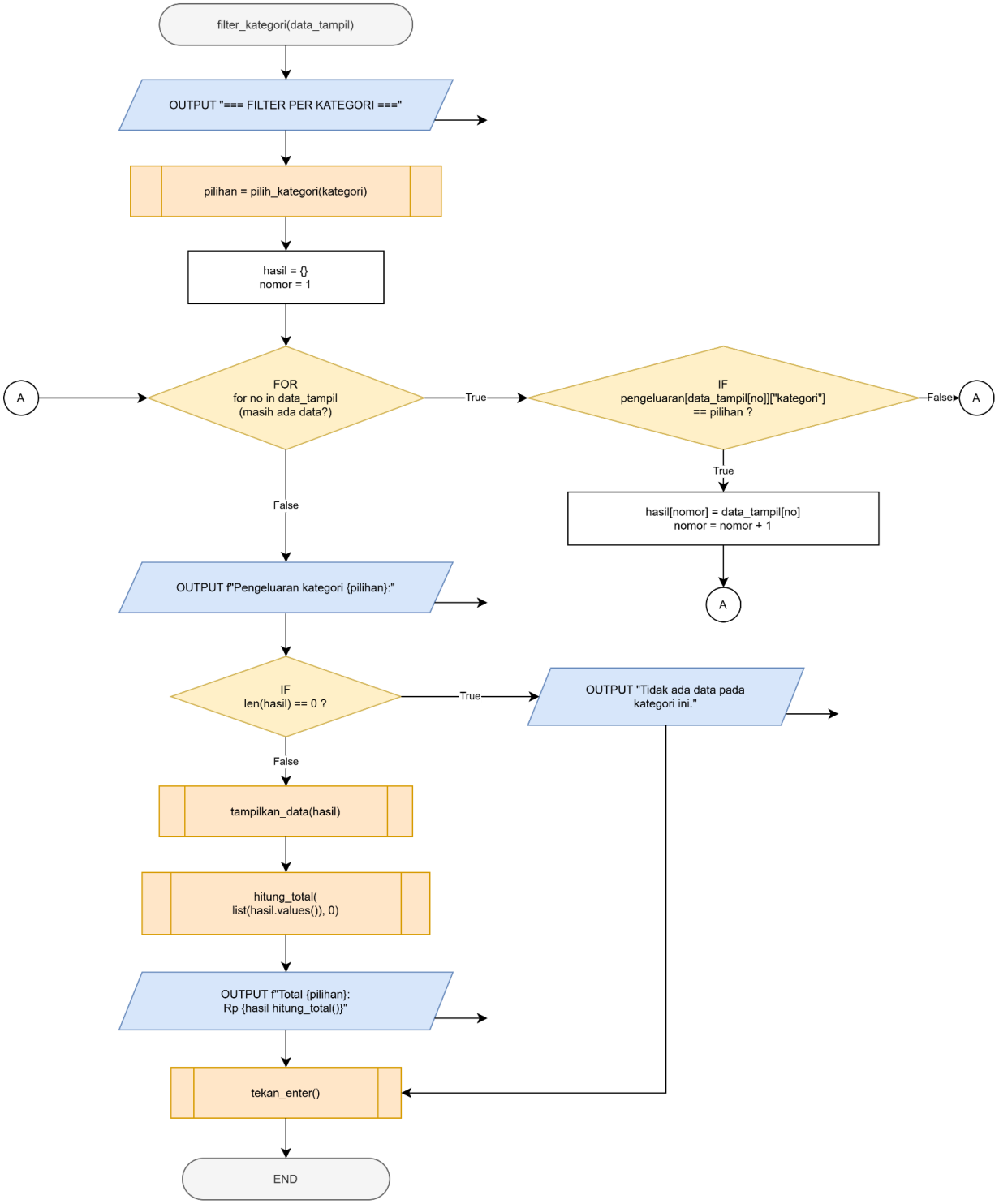
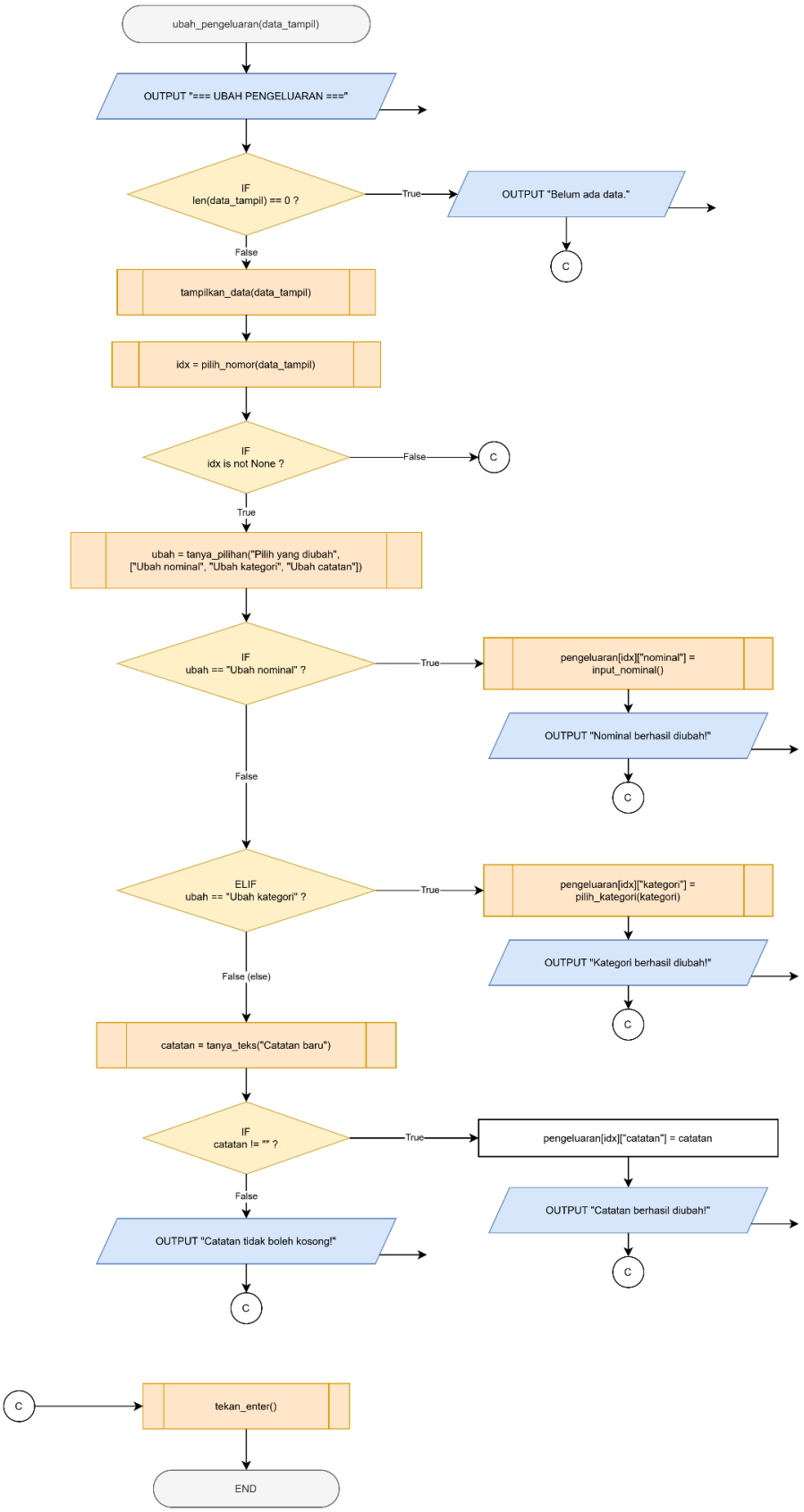


Diagram 1.7. Flowchart tambah_pengeluaran(), lihat_pengeluaran(), dan filter_kategori() (file crud.py)


ubah_pengeluaran(data_tampil)
OUTPUT "=== UBAH PENGELUARAN ==="
IF
len(data_tampil) == 0 ?
True
OUTPUT "Belum ada data."
C
False
tampilkan_data(data_tampil)
idx = pilih_nomor(data_tampil)
IF
idx is not None ?
False
C
True
ubah = tanya_pilihan("Pilih yang diubah",
["Ubah nominal", "Ubah kategori", "Ubah catatan"])
IF
ubah == "Ubah nominal" ?
True
pengeluaran[idx]["nominal"] =
input_nominal()
OUTPUT "Nominal berhasil diubah!"
C
False
ELIF
ubah == "Ubah kategori" ?
True
pengeluaran[idx]["kategori"] =
pilih_kategori(kategori)
OUTPUT "Kategori berhasil diubah!"
C
False (else)
catatan = tanya_teks("Catatan baru")
IF
catatan != "" ?
True
pengeluaran[idx]["catatan"] = catatan
OUTPUT "Catatan berhasil diubah!"
C
False
OUTPUT "Catatan tidak boleh kosong!"
C
C
tekan_enter()
END

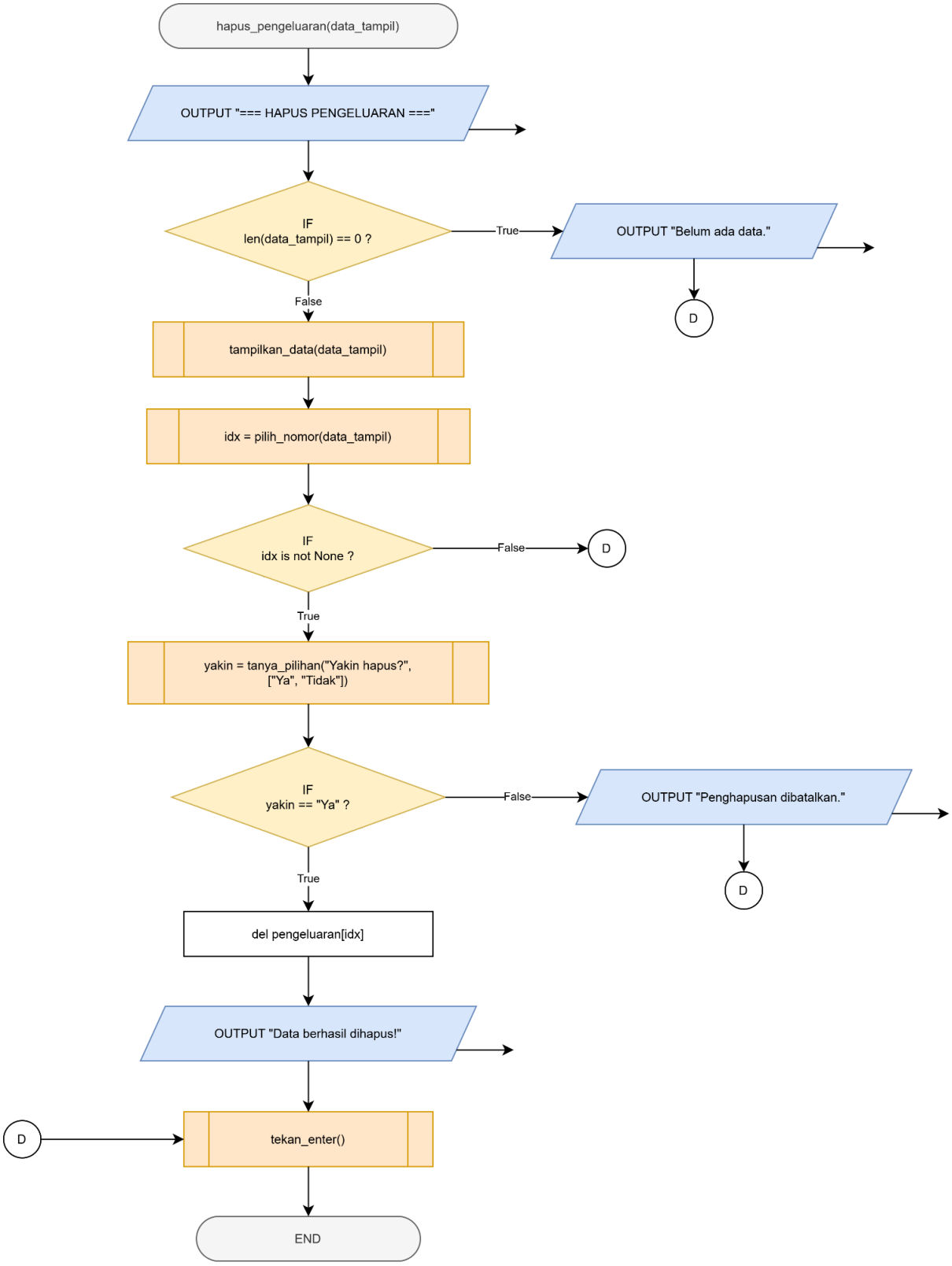


Diagram 1.8. Flowchart ubah_pengeluaran() dan hapus_pengeluaran() (file crud.py)

Penjelasan Algoritma

1. Program dimulai dari file utama (2609106005-BelvaAbhinaya-PT-8.py). File utama mengimpor fungsi dari autentikasi.py, crud.py, dan fungsi_umum.py. Data awal (user, pengeluaran, kategori) disimpan di data.py.
2. main() dijalankan di dalam try, dan laman diisi True.
3. Selama laman = True, program menampilkan menu awal berupa pilihan interaktif inquirer (Login, Register, Keluar). Pengguna memilih dengan tombol panah dan Enter.
4. Pilihan Login. Pengguna punya maksimal 3 kali percobaan. Password diketik tersembunyi dengan inquirer.Password.
   - Kalau username tidak ada atau password salah, percobaan bertambah 1 dan muncul pesan beserta sisa percobaan.
   - Kalau benar, muncul "Login Berhasil", lalu program mengembalikan username dan role.
   - Kalau gagal 3 kali, program kembali ke menu awal.
5. Kalau login berhasil, program masuk ke menu_user().
6. Pilihan Register. Program meminta username baru dan password baru (tersembunyi), lalu memeriksa tiga hal:
   - Username kosong, "Username Tidak Boleh Kosong".
   - Username sudah dipakai, "Username Sudah Terpakai".
   - Password kurang dari 4 karakter, "Password Minimal 4 Karakter".
   - Kalau semuanya aman, akun disimpan dengan role "user".
7. Pilihan Keluar. Muncul "Terima kasih!" dan laman menjadi False. Karena menu berupa daftar pilihan, pengguna tidak dapat memasukkan pilihan di luar daftar.

8. Di menu_user(), program menampilkan enam pilihan menu secara berulang, lalu menyiapkan data_tampil lewat ambil_data_tampil(). Admin melihat semua data, pengguna biasa hanya datanya sendiri.
9. Menu Tambah Pengeluaran. Program meminta isian dan mengulang input kalau salah:
   - Tanggal berformat DD-MM-YYYY (hari 1-31, bulan 1-12, tahun minimal 2026).
   - Nominal berupa angka lebih dari 0.
   - Catatan tidak boleh kosong.
   - Kategori dipilih dari daftar pilihan inquirer. Setelah itu data disimpan.
10. Menu Lihat Semua Pengeluaran. Data ditampilkan dalam tabel PrettyTable (tampilkan_data), lalu total dihitung dengan hitung_total() yang rekursif. Kalau kosong muncul "Belum ada data.".
11. Menu Filter & Total per Kategori. Pengguna memilih kategori, program menampilkan tabel data kategori itu beserta totalnya.
12. Menu Ubah Pengeluaran. Pengguna memilih data dari daftar, lalu memilih yang diubah (nominal, kategori, atau catatan).
13. Menu Hapus Pengeluaran. Pengguna memilih data dari daftar dan dikonfirmasi "Yakin hapus?" dengan pilihan Ya atau Tidak. Data dihapus kalau jawabannya Ya.
14. Menu Logout kembali ke menu awal.
15. Kalau program dihentikan dengan Ctrl+C (termasuk saat berada di prompt inquirer) muncul "Program dihentikan paksa.". Baik berhenti normal maupun paksa, finally menampilkan "Program selesai.".
16. Selesai.

2. Deskripsi Singkat Program

Program ini adalah Sistem Pencatat Keuangan Harian untuk mencatat dan mengelola pengeluaran sehari-hari. Program ini menyediakan register dan login supaya setiap

pengguna punya akun sendiri. Ada dua jenis pengguna, yaitu admin yang dapat melihat dan mengelola seluruh data, dan pengguna biasa yang hanya dapat mengelola datanya sendiri. Percobaan login dibatasi maksimal tiga kali.

Pada Posttest 8, program dari Posttest 7 dilanjutkan dengan dua perubahan utama. Pertama, tampilan data pengeluaran memakai library eksternal PrettyTable sehingga output di terminal berbentuk tabel yang rapi. Kedua, program dipecah menjadi beberapa file berdasarkan fungsionalitasnya (pemrograman modular), sehingga kode lebih mudah dibaca dan dirawat.

program ini juga memakai library inquirer untuk input interaktif. Menu dan pilihan (kategori, data yang diubah atau dihapus, konfirmasi hapus) dipilih dengan tombol panah, sedangkan password diketik tersembunyi dengan inquirer. Password. Library bawaan os tetap dipakai untuk membersihkan layar.

Pembagian file program

File	Isi
main.py	File utama yang dijalankan (main.py). Berisi main() dan menu_user().
data.py	Data program: user, pengeluaran, dan kategori.
fungsi_umum.py	Fungsi bantu: tanya, tanya_teks, tanya_password, tanya_pilihan, input_tanggal, input_nominal, pilih_kategori, ambil_data_tampil, hitung_total, tekan_enter, pilih_nomor.
tampilan.py	Tampilan tabel data pengeluaran dengan PrettyTable: tampilkan_data.
autentikasi.py	Fitur akun: register dan login (password tersembunyi dengan inquirer).
crud.py	Fitur CRUD: tambah, lihat, filter, ubah, dan hapus pengeluaran, serta id_terakhir.

3. Source Code

A. Fungsi Tanpa Parameter

Fungsi tanpa parameter adalah fungsi yang tidak memerlukan data dari luar saat dipanggil dan mengembalikan nilai dengan return. Pada program ini terdapat fungsi input_tanggal() yang mengembalikan tanggal valid dan input_nominal() yang mengembalikan nominal valid. Keduanya mengulang input sampai benar. Kedua fungsi berada di file fungsi_umum.py.

```
def input_tanggal():
    while True:
        try:
            tanggal = tanya_teks("Tanggal (DD-MM-YYYY)")
            bagian = tanggal.split("-")
            if len(bagian) != 3:
                raise ValueError
            hari = int(bagian[0])
            bulan = int(bagian[1])
            tahun = int(bagian[2])
            if not (1 <= hari <= 31 and 1 <= bulan <= 12 and tahun >= 2026):
                raise ValueError
        except ValueError:
            print("Tanggal tidak valid! Contoh: 14-10-2026")
        else:
            return tanggal

def input_nominal():
    while True:
        try:
            nominal = int(tanya_teks("Nominal (angka)"))
            if nominal <= 0:
                raise ValueError
        except ValueError:
            print("Nominal harus angka lebih dari 0!")
        else:
            return nominal
```

B. Fungsi Dengan Parameter

Fungsi dengan parameter menerima nilai (argumen) dari luar untuk diproses. Fungsi tanya_pilihan() menerima pesan dan pilihan lalu mengembalikan pilihan yang dipilih

pengguna. Fungsi pilih_kategori() menerima daftar_kategori dan memanfaatkan tanya_pilihan(). Fungsi ambil_data_tampil() menerima user_aktif dan role lalu mengembalikan dictionary data yang boleh dilihat pengguna tersebut. Semua fungsi ini berada di file fungsi_umum.py.

```
def tanya_pilihan(pesan, pilihan):
    return tanya(inquirer.List("x", message=pesan, choices=pilihan))


def pilih_kategori(daftar_kategori):
    return tanya_pilihan("Pilih kategori", list(daftar_kategori))


def ambil_data_tampil(user_aktif, role):
    data_tampil = {}
    nomor_urut = 1
    for id_data in pengeluaran:
        if role == "admin" or pengeluaran[id_data]["user"] == user_aktif:
            data_tampil[nomor_urut] = id_data
            nomor_urut += 1
    return data_tampil
```

C. Fungsi Rekursif

Fungsi Rekursif adalah fungsi yang memecahkan masalah dengan cara memanggil dirinya sendiri hingga kondisi tertentu, tetapi dengan argumen yang berbeda, untuk menyelesaikan sub-masalah yang lebih kecil. Fungsi rekursif punya dua syarat, yaitu Basis (Base Case) sebagai kondisi fungsi berhenti agar tidak terjadi perulangan tak terbatas (RecursionError), dan Rekursi (Recursive Case) yaitu proses fungsi memanggil dirinya sendiri dengan input yang makin mendekati base case. Pada program ini fungsi hitung_total() di file fungsi_umum.py menjumlahkan nominal pengeluaran satu per satu. Base case-nya adalah i == len(daftar_id) yang mengembalikan 0, sedangkan recursive case-nya memanggil hitung_total(daftar_id, i + 1).

```
def hitung_total(daftar_id, i):
    # Base Case: kondisi fungsi berhenti
```

```
    if i == len(daftar_id):
        return 0
    # Recursive Case: fungsi memanggil dirinya sendiri
    return pengeluaran[daftar_id[i]]["nominal"] + hitung_total(daftar_id, i
+ 1)
```

D. Prosedur

Prosedur adalah fungsi yang menjalankan blok kode tanpa mengembalikan nilai (tidak memakai return). Contohnya prosedur tekan_enter() di file fungsi_umum.py dan tampilkan_data() di file tampilan.py yang hanya mencetak tampilan ke layar.

```
def tekan_enter():
    input("Tekan Enter untuk lanjut...")


def tampilkan_data(data_tampil):
    tabel = PrettyTable()
    tabel.field_names = ["No", "Tanggal", "Kategori", "Nominal", "Catatan",
"User"]
    tabel.align["Catatan"] = "l"
    for no in data_tampil:
        d = pengeluaran[data_tampil[no]]
        tabel.add_row([no, d['tanggal'], d['kategori'], f"Rp
{d['nominal']}", d['catatan'], d['user']])
    print(tabel)
```

E. Fitur Input Data

Fitur input digunakan untuk menerima data dari pengguna, seperti username, password, tanggal, nominal, catatan, kategori, dan pilihan menu. Input dibungkus dalam fungsi tanya_teks(), tanya_password(), dan tanya_pilihan() yang memakai library inquirer.

```
def tanya(pertanyaan):
    jawaban = inquirer.prompt([pertanyaan])
    if jawaban is None:
        raise KeyboardInterrupt
    return jawaban["x"]
```

```
def tanya_teks(pesan):
    return tanya(inquirer.Text("x", message=pesan))

def tanya_password(pesan):
    return tanya(inquirer.Password("x", message=pesan))
```

F. Fitur Percabangan (IF/ELIF/ELSE)

Percabangan digunakan untuk menentukan tindakan berdasarkan kondisi, yaitu memeriksa pilihan menu dan memanggil fungsi atau prosedur yang sesuai.

```
        if menu == "Tambah Pengeluaran":
            tambah_pengeluaran(user_aktif)
        elif menu == "Lihat Semua Pengeluaran":
            lihat_pengeluaran(data_tampil)
        elif menu == "Filter & Total per Kategori":
            filter_kategori(data_tampil)
        elif menu == "Ubah Pengeluaran":
            ubah_pengeluaran(data_tampil)
        elif menu == "Hapus Pengeluaran":
            hapus_pengeluaran(data_tampil)
        else:
            break
```

G. Fitur Perulangan (FOR dan WHILE)

Perulangan while digunakan untuk menampilkan menu berulang dan mengulang input sampai valid. Perulangan for digunakan untuk membaca data pengeluaran, misalnya pada ambil_data_tampil() dan tampilkan_data().def main():

```
    laman = True
    while laman:
        os.system('cls')
        print("=== SISTEM PENCATAT KEUANGAN HARIAN ===")
        pilihan = tanya_pilihan("Pilih Menu", ["Login", "Register",
"Keluar"])

        if pilihan == "Login":
            user_aktif, role = login()
            if user_aktif != "":
                menu_user(user_aktif, role)
```

```
        elif pilihan == "Register":
            register()
        else:
            print("Terima kasih!")
            laman = False
```

H. Fitur Dictionary, Tuple, dan Nested Dictionary

Dictionary menyimpan data berpasangan key dan value. Data akun memakai username sebagai key dengan value berisi password dan role. Data pengeluaran adalah nested dictionary dengan id transaksi sebagai key dan value berisi pemilik, tanggal, nominal, catatan, dan kategori. Tuple dipakai untuk daftar kategori yang tidak berubah. Ketiganya disimpan di file data.py. Variabel user, pengeluaran, dan kategori adalah variabel global karena dibuat di luar fungsi sehingga dapat diakses oleh fungsi manapun, sedangkan variabel yang dibuat di dalam fungsi (misalnya percobaan pada login()) adalah variabel lokal yang hanya dapat diakses di dalam fungsi tempat ia berada.

```
user = {
    "admin": {"password": "admin123", "role": "admin"},
    "belva": {"password": "belva005", "role": "user"}
}
pengeluaran = {}
kategori = ("ATK", "Makanan", "Transportasi", "Hiburan", "Lainnya")
```

I. Fitur CRUD

CRUD adalah bagian utama program dan berada di file crud.py. Create menambah pasangan key dan value baru ke dictionary pengeluaran. Read menampilkan riwayat dan total. Update mengubah value pada key tertentu. Delete menghapus data dengan del.

1). Create

```
id_terakhir += 1
    pengeluaran[id_terakhir] = {
        "user": user_aktif,
        "tanggal": tanggal,
        "nominal": nominal,
        "catatan": catatan,
        "kategori": kat
    }
    print("Data berhasil ditambahkan!")
    tekan_enter()
```

2). Read

```
def lihat_pengeluaran(data_tampil):
    os.system('cls')
    print("=== RIWAYAT PENGELUARAN ===")
    if len(data_tampil) == 0:
        print("Belum ada data.")
    else:
        tampilkan_data(data_tampil)
        total = hitung_total(list(data_tampil.values()), 0)
        print(f"Total keseluruhan: Rp {total}")
    tekan_enter()
```

3). Update

```
            ubah = tanya_pilihan("Pilih yang diubah",
                                 ["Ubah nominal", "Ubah kategori", "Ubah
catatan"])
            if ubah == "Ubah nominal":
                pengeluaran[idx]["nominal"] = input_nominal()
                print("Nominal berhasil diubah!")
            elif ubah == "Ubah kategori":
                pengeluaran[idx]["kategori"] = pilih_kategori(kategori)
                print("Kategori berhasil diubah!")
            else:
                catatan = tanya_teks("Catatan baru")
                if catatan != "":
                    pengeluaran[idx]["catatan"] = catatan
                    print("Catatan berhasil diubah!")
                else:
                    print("Catatan tidak boleh kosong!")
    tekan_enter()
```

4). Delete

```
def hapus_pengeluaran(data_tampil):
    os.system('cls')
    print("=== HAPUS PENGELUARAN ===")
    if len(data_tampil) == 0:
        print("Belum ada data.")
    else:
        tampilkan_data(data_tampil)
        idx = pilih_nomor(data_tampil)
        if idx is not None:
            yakin = tanya_pilihan("Yakin hapus?", ["Ya", "Tidak"])
            if yakin == "Ya":
                del pengeluaran[idx]
                print("Data berhasil dihapus!")
            else:
                print("Penghapusan dibatalkan.")
    tekan_enter()
```

J. Fitur Error Handling

Error handling mencegah program berhenti mendadak akibat input yang salah. Blok try berisi kode yang mungkin menimbulkan error, blok except menangani error tertentu (ValueError, KeyError), blok else dijalankan jika tidak ada error, dan blok finally selalu dijalankan. Kata kunci raise dipakai untuk membuat error secara manual saat validasi gagal. Pada fungsi tanya(), jika pengguna menekan Ctrl+C saat prompt inquirer, hasilnya None sehingga program melempar KeyboardInterrupt yang ditangkap di file utama. Fungsi register() dan login() berada di autentikasi.py.

```
def tanya(pertanyaan):
    jawaban = inquirer.prompt([pertanyaan])
    if jawaban is None:
        raise KeyboardInterrupt
    return jawaban["x"]


def register():
    os.system('cls')
    print("=== REGISTER ===")
    username_baru = tanya_teks("Masukkan Username Baru")
```

```
    password_baru = tanya_password("Masukkan Password Baru")
    try:
        if username_baru == "":
            raise ValueError("Username Tidak Boleh Kosong")
        if username_baru in user:
            raise ValueError("Username Sudah Terpakai")
        if len(password_baru) < 4:
            raise ValueError("Password Minimal 4 Karakter")
    except ValueError as e:
        print(e)
    else:
        user[username_baru] = {"password": password_baru, "role": "user"}
        print("Register berhasil! Silakan login.")
    tekan_enter()

def login():
    percobaan = 0
    while percobaan < 3:
        os.system('cls')
        print("=== LOGIN ===")
        username = tanya_teks("Username")
        password = tanya_password("Password")
        try:
            if user[username]["password"] != password:
                raise ValueError
        except (KeyError, ValueError):
            percobaan += 1
            print(f"Username/password salah! Sisa percobaan: {3 -
percobaan}")
            tekan_enter()
        else:
            print("Login Berhasil")
            tekan_enter()
            return username, user[username]["role"]
    print("Gagal login 3 kali, kembali ke menu awal.")
    tekan_enter()
    return "", ""


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("Program dihentikan paksa.")
    finally:
        print("Program selesai.")
```

K. Fitur Clear Terminal dengan Library OS

Library os digunakan untuk membersihkan terminal setiap berpindah menu. Perintah os.system('cls') digunakan pada terminal Windows.

```
import os
os.system('cls')
```

L. Fitur Output (Print dan F-String)

Fitur output menampilkan judul menu, riwayat, total, pesan validasi, dan pesan keberhasilan. F-string menampilkan nilai variabel langsung di dalam teks.

```
        print(f"=== MENU {role.upper()} - {user_aktif} ===")
        menu = tanya_pilihan("Pilih menu", [
            "Tambah Pengeluaran",
            "Lihat Semua Pengeluaran",
            "Filter & Total per Kategori",
            "Ubah Pengeluaran",
            "Hapus Pengeluaran",
            "Logout"
        ])

        print("Program dihentikan paksa.")
        print("Register berhasil! Silakan login.")
        print("Gagal login 3 kali, kembali ke menu awal.")
```

M. Fitur Pemrograman Modular

Pemrograman modular adalah teknik merancang program dengan cara memecah satu kode program yang besar dan panjang menjadi beberapa file Python (.py) yang lebih kecil, independen, serta saling terhubung. Setiap file berperan sebagai modul yang fokus menangani satu aspek fungsionalitas tertentu. Manfaatnya adalah keterbacaan (readability), keterpeliharaan (maintainability), penggunaan kembali (reusability), dan kemudahan kolaborasi tim. Semua file diletakkan dalam satu folder proyek yang sama dan hanya satu file utama yang dijalankan. Cara mengimpor file buatan sendiri sama persis seperti mengimpor library, yaitu memakai nama file tanpa akhiran .py.

```
import os
from autentikasi import login, register
from crud import (tambah_pengeluaran, lihat_pengeluaran, filter_kategori,
                  ubah_pengeluaran, hapus_pengeluaran)
from fungsi_umum import ambil_data_tampil, tanya_pilihan
```

N. Fitur Library PrettyTable

PrettyTable adalah library eksternal (diinstal dengan pip install prettytable) untuk menampilkan data dalam bentuk tabel bergaris di terminal. field_names menentukan nama kolom, add_row menambah satu baris data, dan align mengatur perataan teks pada kolom tertentu. Pada program ini PrettyTable dipakai di file tampilan.py untuk menampilkan data pengeluaran.

```
from prettytable import PrettyTable
from data import pengeluaran

def tampilkan_data(data_tampil):
    tabel = PrettyTable()
    tabel.field_names = ["No", "Tanggal", "Kategori", "Nominal", "Catatan", "User"]
    tabel.align["Catatan"] = "l"
    for no in data_tampil:
        d = pengeluaran[data_tampil[no]]
        tabel.add_row([no, d['tanggal'], d['kategori'], f"Rp {d['nominal']}",
d['catatan'], d['user']])
    print(tabel)
```

O. Fitur Library Inquirer

Inquirer adalah library eksternal (diinstal dengan pip install inquirer) untuk membuat prompt interaktif di terminal. inquirer.Text dipakai untuk input teks, inquirer.Password untuk input yang disembunyikan, dan inquirer.List untuk daftar pilihan yang dipilih dengan tombol panah. Pada program ini ketiganya dibungkus dalam fungsi tanya_teks(), tanya_password(), dan tanya_pilihan() di file fungsi_umum.py, sehingga menu, kategori, dan konfirmasi tidak perlu diketik manual dan kesalahan input pilihan dapat dihindari. Fungsi pilih_nomor() juga memakai inquirer.List dengan pasangan (label tampilan, id data) sehingga pengguna memilih data langsung dari daftar.

```python
def tanya(pertanyaan):
    jawaban = inquirer.prompt([pertanyaan])
    if jawaban is None:
        raise KeyboardInterrupt
    return jawaban["x"]

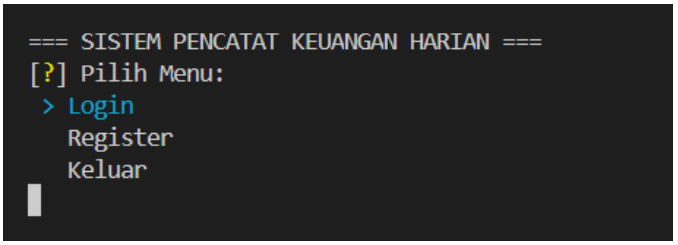
def tanya_teks(pesan):
    return tanya(inquirer.Text("x", message=pesan))

def tanya_password(pesan):
    return tanya(inquirer.Password("x", message=pesan))

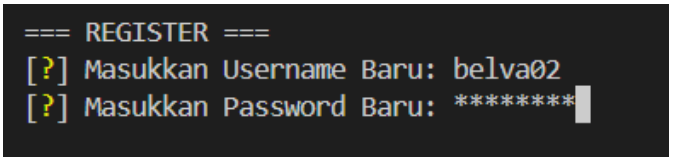
def tanya_pilihan(pesan, pilihan):
    return tanya(inquirer.List("x", message=pesan, choices=pilihan))

def pilih_nomor(data_tampil):
    pilihan = []
    for no in data_tampil:
        d = pengeluaran[data_tampil[no]]
        label = f"{no}. {d['tanggal']} | {d['kategori']} | Rp {d['nominal']}
| {d['catatan']}"
        pilihan.append((label, data_tampil[no]))
    return tanya_pilihan("Pilih data", pilihan)
```

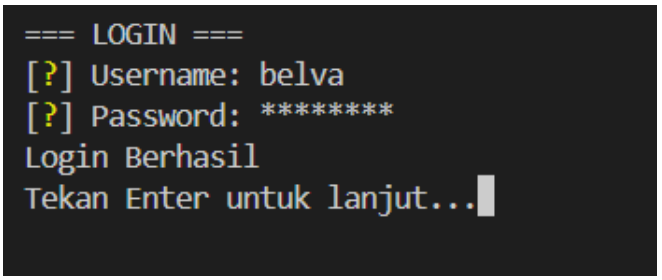
4. Hasil Output



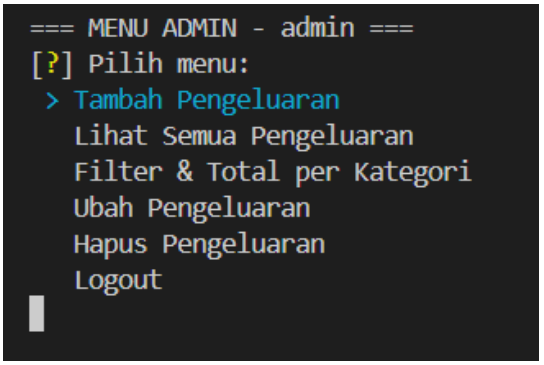
Gambar 4.1 Tampilan Menu Utama (pilihan inquirer)



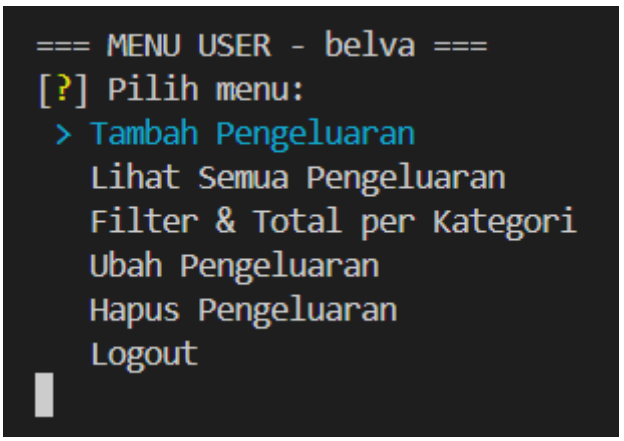
Gambar 4.2 Tampilan Register (password tersembunyi)



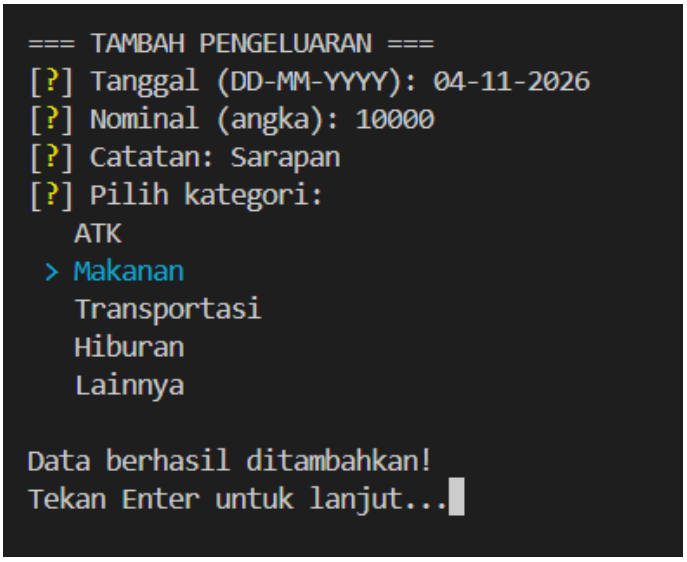
Gambar 4.3 Tampilan Jika Login Berhasil



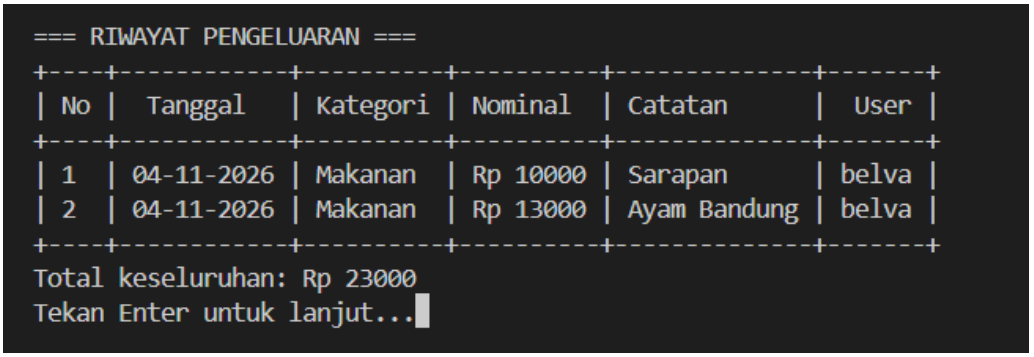
Gambar 4.4 Tampilan Menu Admin



Gambar 4.5 Tampilan Menu Pengguna Biasa



Gambar 4.6 Tampilan Menu Tambah Pengeluaran



Gambar 4.7 Tampilan Menu Riwayat Pengeluaran (tabel PrettyTable dan total)

```
=== FILTER PER KATEGORI ===
[?] Pilih kategori:
   ATK
 > Makanan
   Transportasi
   Hiburan
   Lainnya


Pengeluaran kategori Makanan:
+----+------------+----------+----------+--------------+-------+
| No |  Tanggal   | Kategori | Nominal  | Catatan      |  User |
+----+------------+----------+----------+--------------+-------+
| 1  | 04-11-2026 | Makanan  | Rp 10000 | Sarapan      | belva |
| 2  | 04-11-2026 | Makanan  | Rp 13000 | Ayam Bandung | belva |
+----+------------+----------+----------+--------------+-------+
Total Makanan: Rp 23000
Tekan Enter untuk lanjut...
```

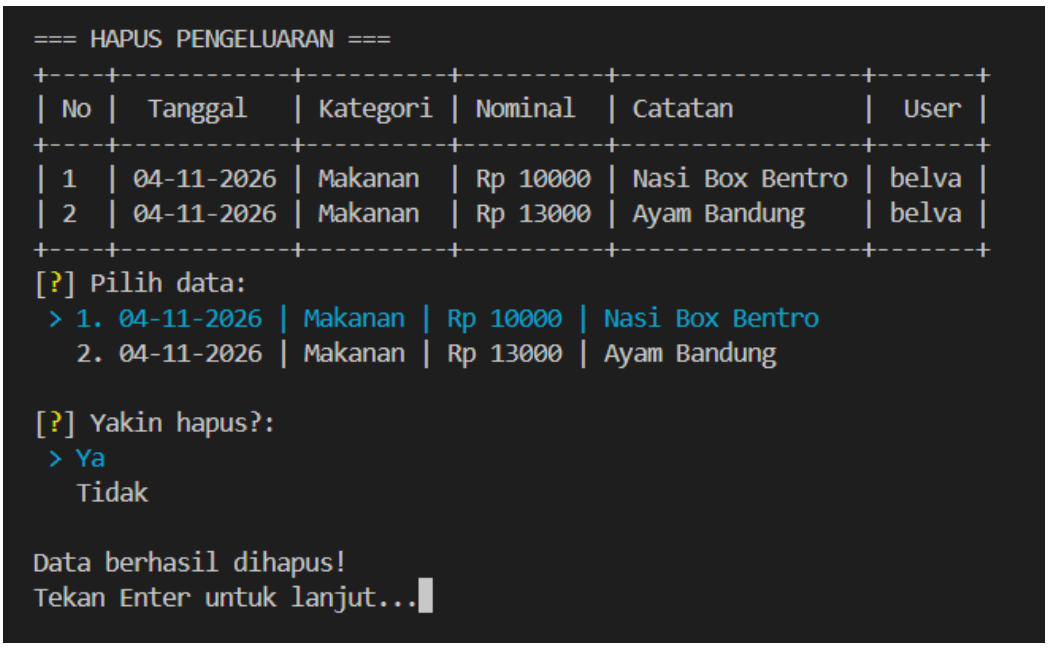
Gambar 4.8 Tampilan Menu Filter dan Total per Kategori

```
=== UBAH PENGELUARAN ===
+----+------------+----------+----------+--------------+-------+
| No |  Tanggal   | Kategori | Nominal  | Catatan      |  User |
+----+------------+----------+----------+--------------+-------+
| 1  | 04-11-2026 | Makanan  | Rp 10000 | Sarapan      | belva |
| 2  | 04-11-2026 | Makanan  | Rp 13000 | Ayam Bandung | belva |
+----+------------+----------+----------+--------------+-------+
[?] Pilih data:
 > 1. 04-11-2026 | Makanan | Rp 10000 | Sarapan
   2. 04-11-2026 | Makanan | Rp 13000 | Ayam Bandung

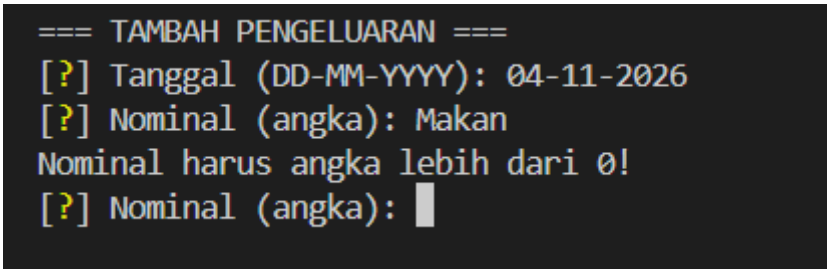
[?] Pilih yang diubah:
   Ubah nominal
   Ubah kategori
 > Ubah catatan

[?] Catatan baru: Nasi Box Bentro
Catatan berhasil diubah!
Tekan Enter untuk lanjut...
```

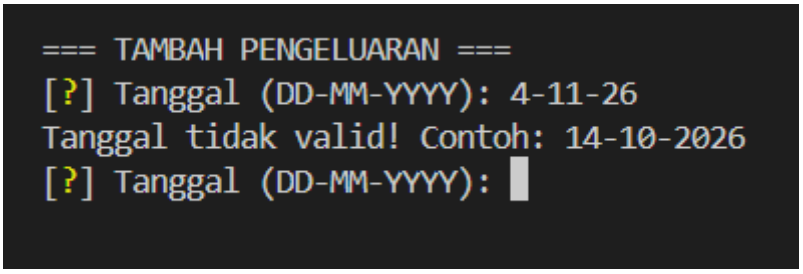
Gambar 4.9 Tampilan Menu Ubah Pengeluaran



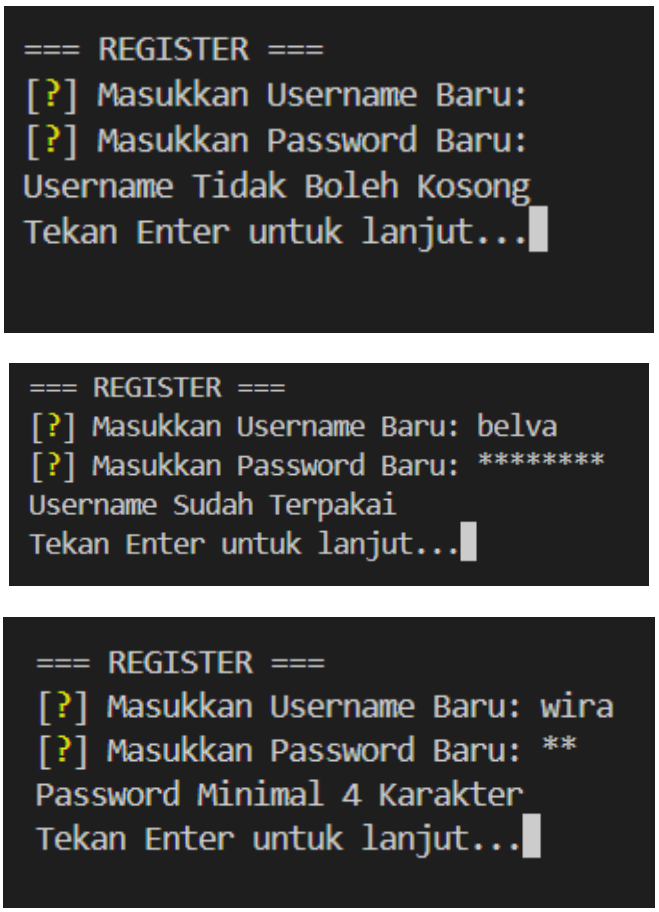
Gambar 4.10 Tampilan Menu Hapus Pengeluaran



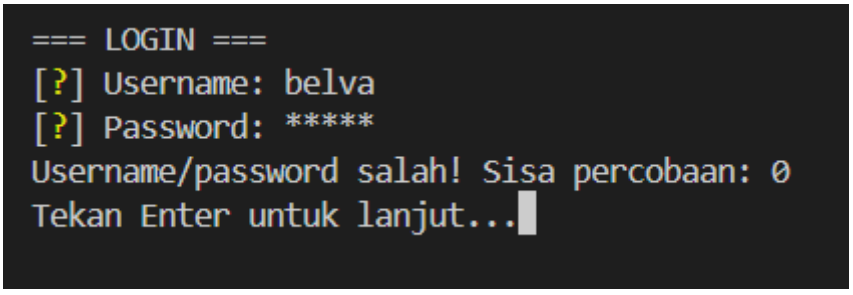
Gambar 4.11 Output Error Handling Nominal Bukan Angka



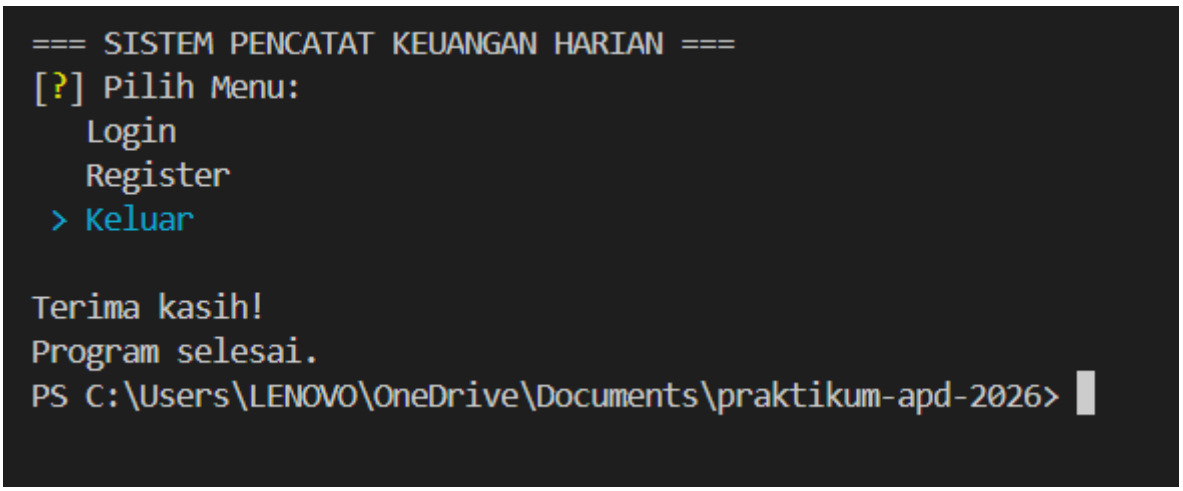
Gambar 4.12 Output Error Handling Format Tanggal Salah



Gambar 4.13 Output Validasi Register



Gambar 4.14 Output Gagal Login 3 Kali



Gambar 4.15 Output Keluar dari Program

5. Git

5.1 Git Add

Perintah git add digunakan untuk menambahkan file baru atau perubahan file ke dalam staging area Git sebelum disimpan melalui commit. Perintah ini dapat digunakan untuk memilih file tertentu atau menambahkan semua perubahan.



Gambar 5.1 Git Add

5.2 Git Commit

Perintah git commit digunakan untuk menyimpan perubahan yang sudah dimasukkan ke staging area sebagai sebuah riwayat perubahan di repositori lokal. Setiap commit dapat diberi pesan untuk menjelaskan perubahan yang dilakukan.

```
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026> git commit -m "POSTTEST 8"
[main 7a827fa] POSTTEST 8
 14 files changed, 376 insertions(+)
 create mode 100644 praktikum-apd-2026/A1/testing.py
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8.docx
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/__pycache__/autentikasi.cpython-314.pyc
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/__pycache__/crud.cpython-314.pyc
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/__pycache__/data.cpython-314.pyc
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/__pycache__/fungsi_umum.cpython-314.pyc
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/__pycache__/tampilan.cpython-314.pyc
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/__pycache__/utilitas.cpython-314.pyc
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/autentikasi.py
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/crud.py
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/data.py
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/fungsi_umum.py
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/main.py
 create mode 100644 praktikum-apd-2026/posttest/posttest-apd-8/2609106005-BelvaAbhinaya-PT-8/tampilan.py
```

Gambar 5.2 Git Commit

5.3 Git Push

Perintah git push digunakan untuk mengirimkan commit dari repositori lokal ke repositori jarak jauh, misalnya GitHub. Dengan perintah ini, perubahan program dapat diunggah dan disimpan di repositori online.

```
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026> git push
info: please complete authentication in your browser...
Enumerating objects: 26, done.
Counting objects: 100% (26/26), done.
Delta compression using up to 16 threads
Compressing objects: 100% (21/21), done.
Writing objects: 100% (22/22), 27.84 KiB | 1.55 MiB/s, done.
Total 22 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To https://github.com/BelvaAbhinaya005/praktikum-apd-2026.git
   7c3e3f7..7a827fa  main -> main
PS C:\Users\LENOVO\OneDrive\Documents\praktikum-apd-2026>
```

Gambar 5.3 Git Push